Zen User Guide

Zen — User Guide

Zen is a dark-themed web console for Roku QA / test-automation. From one UI you can manage device configurations, build and run test profiles, run individual Roku tests locally, browse and cache TestRail results, run AI root-cause analysis on failed logs (and keep a history), talk to an AI assistant with access to Roku internal data (JIRA / Confluence / TestRail / GitLab), transfer files and run shell commands / interactive SSH sessions, keep a library of reference links and docs, trigger and track QA Hub donut runs, convert devices to Connected / Unconnected Mode, see everything at a glance on the Dashboard, and — as an admin — configure the whole system.

Live instance: http://10.64.16.16:9898/ http://10.64.26.234:9898/ **Backend API:** proxied under `/api` to the FastAPI service on port 9888.

> **Default admin account:** `admin` / `Zaq1@wsx`. Use **Admin Login** (top-right) to unlock the **Settings** tab (§14). Without admin login, Settings is hidden and the app runs in normal (non-admin) mode.

Contents

Contents
1. Getting Started
   1.1 Layout
   1.2 Navigation tabs
   1.3 Theme
   1.4 Admin Login
2. Device Tab
   2.1 Browsing devices
   2.2 Viewing / editing a device
   2.3 Adding a device
3. Profile Tab
   3.1 Managing profiles
   3.2 Auto-match a device (Add / Edit Profile)
   3.3 Split a profile across devices
   3.4 Add Profile with RunPlan (generate from TestRail)
4. Run Case Tab
   4.1 Running a profile
   4.2 Running Cases (local processes)
   4.3 Running Suites (QA Hub)
   4.4 Run Results (cache & reports)
   4.5 Analyze a result
5. Run Case in local Tab
   5.1 Running a test

- 5.2 Live log
- 5.3 Test Logs
- 6. Analyze History Tab
- 7. TestRail Case Tab
  - 7.1 Plans
  - 7.2 Searching cases
  - 7.3 Feeding the profile generator
- 8. Prompt Tab (AI Assistant)
  - 8.1 Agent card — model info & switching
  - 8.2 Sessions (multi-turn chat)
  - 8.3 Skills
  - 8.4 Prompt Run
  - 8.5 Search card
- 9. Reference Tab
- 10. File Transfer Tab
  - 10.1 File Transfer sub-tab
  - 10.2 Cmdline sub-tab
- 11. Dashboard Tab
  - 11.1 Controls
- 12. Donuts Tab
  - 12.1 Trigger a donut run
  - 12.2 Runs table
  - 12.3 Results
  - 12.4 Remove a run
- 13. Activate Tab
  - 13.1 Start a conversion
  - 13.2 Precheck result
  - 13.3 Live Logs
  - 13.4 Conversion Jobs
- 14. Settings Tab (admin only)
  - 14.1 Tab Settings
  - 14.2 Agent Settings
  - 14.3 Environment Settings
  - 14.4 Activate Account
- 15. Tips & Troubleshooting

1. Getting Started

Open http://10.64.16.16:9898/ in a browser (Chrome/Edge recommended — see the note on Save/Copy below). No login is required for normal use; admin login is only needed for Settings.

1.1 Layout

- **Header bar** (sticky at top) — the brand **Zen**, a theme switcher, and the **Admin Login / Admin Logout** button.
- **Left navigation (Sider)** (sticky) — the tab menu; it scrolls internally if long. The **Settings** entry (admin-only) and the **Collapse** toggle stay pinned at the bottom of the viewport.
- **Content area** — a rounded card showing the current page; this is the part that scrolls.

1.2 Navigation tabs

Default order below. Order and visibility are configurable in **Settings → Tab Settings** (§14.1). **Settings** itself only appears after Admin Login.

Tab	Purpose
Dashboard	Default landing page: one summary card per tab. Click a card to open that tab (§14).
Device	Manage device configuration files (the TVs / Roku devices under test).
Profile	Manage test profiles and generate them from cached TestRail data.
Run Case	Run test profiles; watch local runner processes and QA Hub suites; review, export, and AI-analyze results.
Run Case in local	Run a single Roku test locally with a live log tail; browse, filter, and AI-analyze past logs.
Analyze History	Browse, filter, view, and delete the AI root-cause analyses saved from Run Case / Run Case in local.
TestRail Case	Manage TestRail plans and search their cases by status (results are cached).
Prompt	AI assistant (RAG) over Roku internal data, with multi-turn chat sessions and reusable prompt Skills.
Reference	A library of reference links and uploaded Markdown docs.
File Transfer	Browse/upload/transfer files, serve them over HTTP, and run shell

	commands / interactive SSH sessions.
Donuts	Trigger QA Hub donut runs on a device and follow, abort or review them (§12).
Activate	Convert devices to Connected / Unconnected Mode, with live logs (§13).
Settings *(admin only)*	Configure tab visibility/order, agent settings, and account credentials/paths.

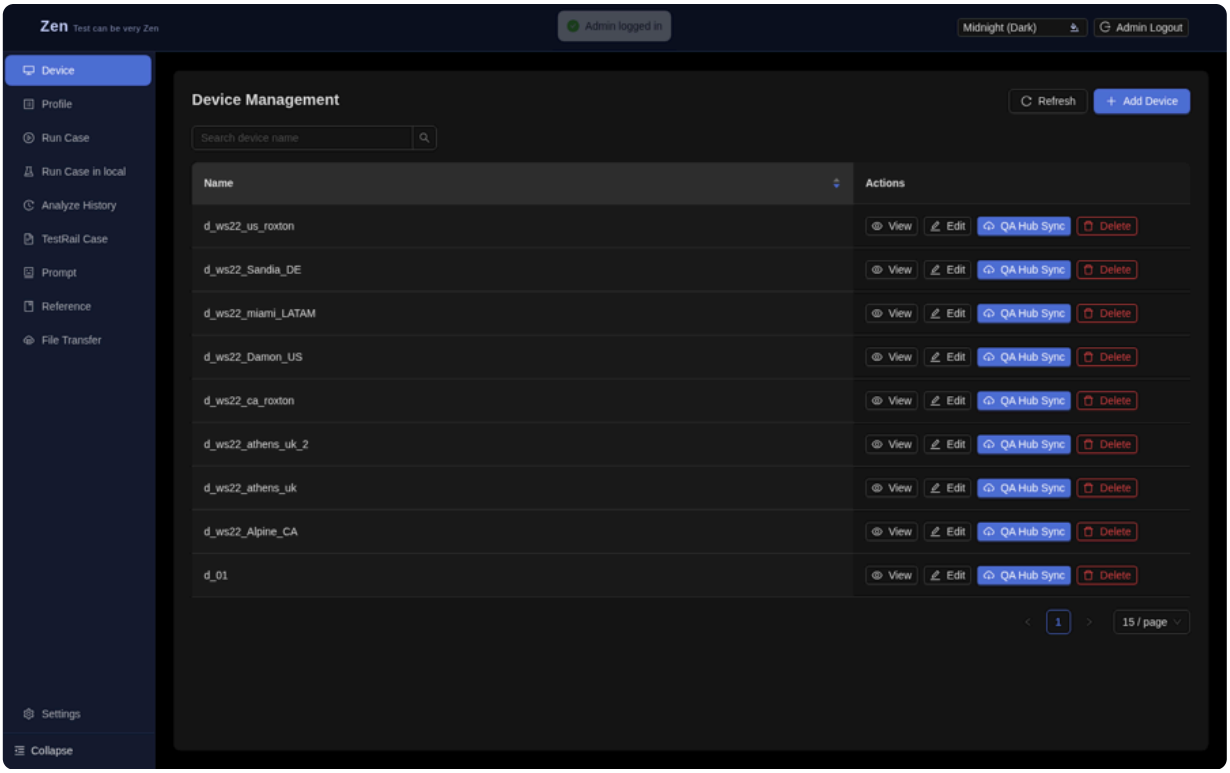
1.3 Theme

Use the header theme switcher to pick from 19 themes grouped into **Dark** (10 — Midnight is the default) and **Light** (9). Your choice is remembered in the browser.

1.4 Admin Login

Click **Admin Login** (top-right), enter the admin username and password (default `admin` / `Zaq1@wsx`), and submit. On success the **Settings** tab appears in the sider and the button becomes **Admin Logout**. The admin state lives in the browser session only (a page reload returns to non-admin). Credentials are verified server-side (bcrypt); the password is never stored or echoed.

2. Device Tab



Lists every device configuration file. Each device is a JSON file describing a TV/Roku under test (IP, brand, power strip, QA bot, serial console, etc.).

2.1 Browsing devices

The table shows one **Name** per device (sortable). Use **Search** to filter; paging at the bottom; **Refresh** reloads from the server.

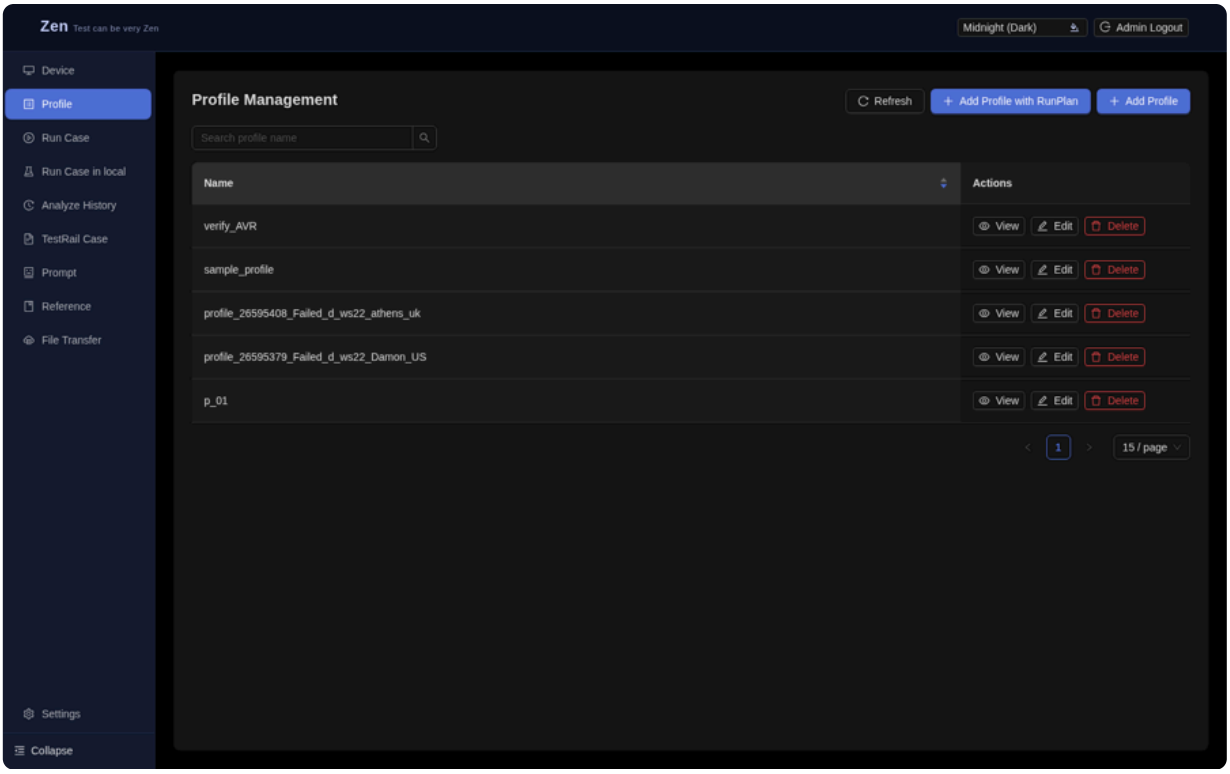
2.2 Viewing / editing a device

- **View** — read-only JSON with syntax highlighting.
- **Edit** — editable JSON editor; validated as JSON before **Save**.
- **Sync** — refresh the device config from QA Hub.
- **Delete** — remove after confirmation.

2.3 Adding a device

Click **Add Device**, enter a **Name** (without `.json`; becomes the device key used in profiles, e.g. `d_ws22_Sandia_DE`), fill the **Content** (JSON — needs at least `ip_address`, `brand_name` and the ids under a `roku` object), and **Save**.

3. Profile Tab



A profile is a JSON file describing what to run: which cases, on which devices, in what order.

3.1 Managing profiles

Browse / Search / View / Edit / Delete / Add work like the Device tab. The table also shows **Description**, **Execution mode** and **Total tasks** per profile.

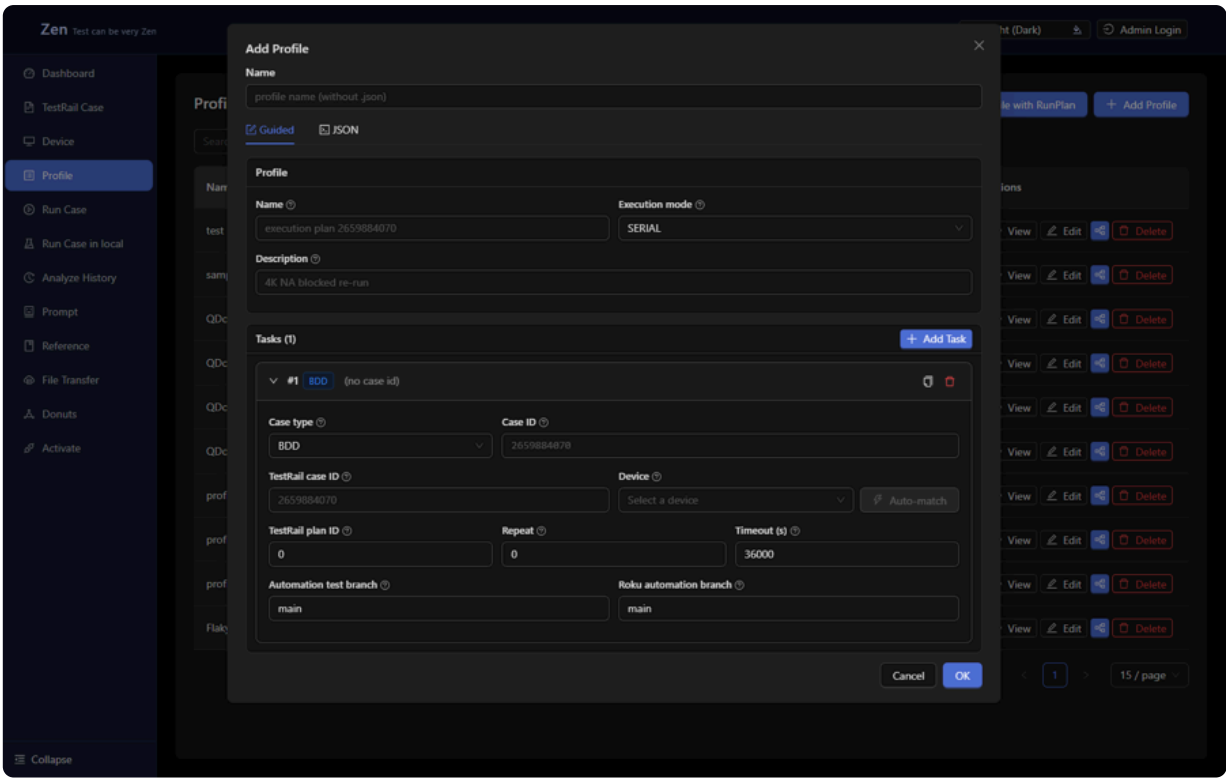
The Add / Edit / View dialog has **two tabs over the same content**, so you can switch freely without losing edits:

- **Guided** (default) — a form: profile *Name*, *Description* and *Execution mode* (SERIAL / PARALLEL), then one collapsible card per task showing *Case type* (BDD / UT / NEW), *Case ID*, *TestRail case ID*, *Device*, *TestRail plan ID*, *Repeat*, both branches and *Timeout*. Each task card can be duplicated or removed, and **Add Task** appends a new one. Every field has a hover hint with an example value.
- **JSON** — the raw editor with syntax highlighting; hovering a key shows what it accepts. Invalid JSON is reported and blocks Save.

> ✅ A task now holds exactly **one** case configuration (key `config`). The old per-task `configs` list and the `execution_mode` / `stop_if_success` / `print_console_message` keys are no longer edited here and are dropped from anything Zen saves; older hand-written profiles still run.

3.2 Auto-match a device (Add / Edit Profile)

When you build a profile by hand you must pick a device for every task — and the device has to actually satisfy that case's conditions (right region, linked or unlinked, the right peripherals wired up). **Auto-match** does that for you: give it the TestRail case ID and it picks a device that fits.



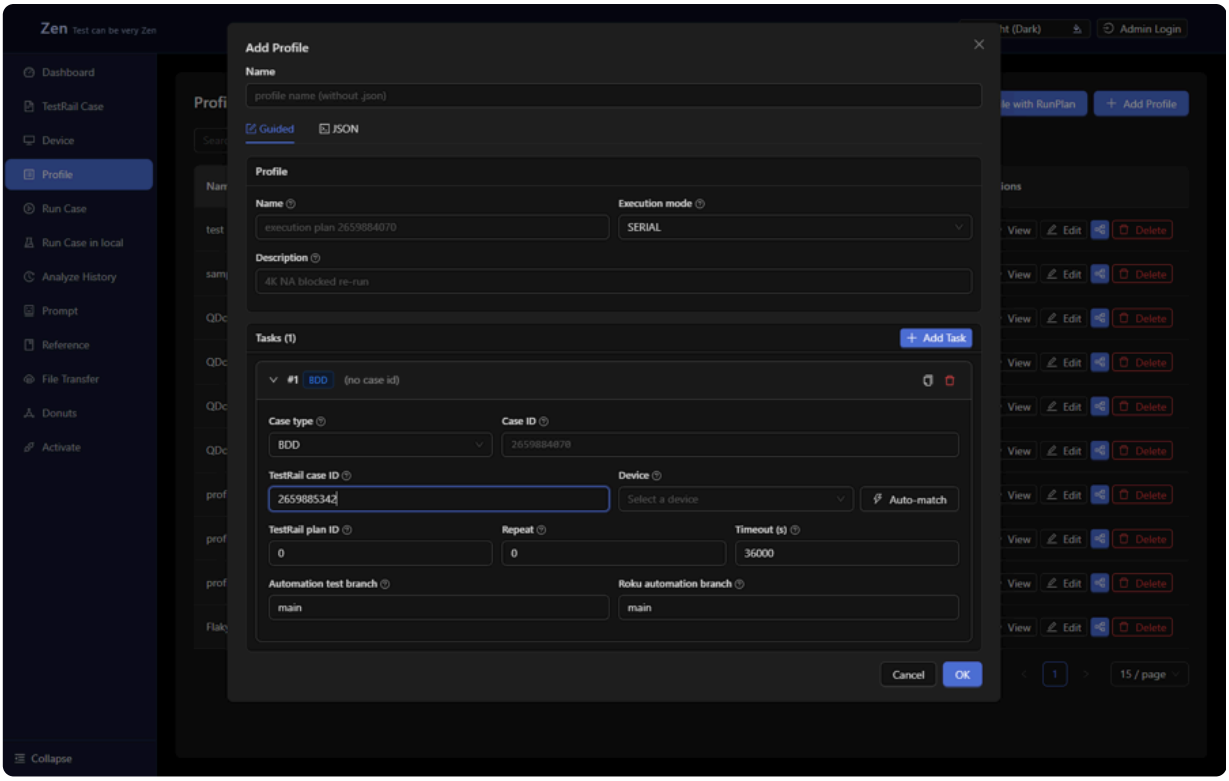
3.2.1 Where it is

Click **Add Profile** (or **Edit** on an existing profile) and stay on the **Guided** tab. Expand a task — the **Auto-match** button sits directly to the right of the **Device** dropdown.

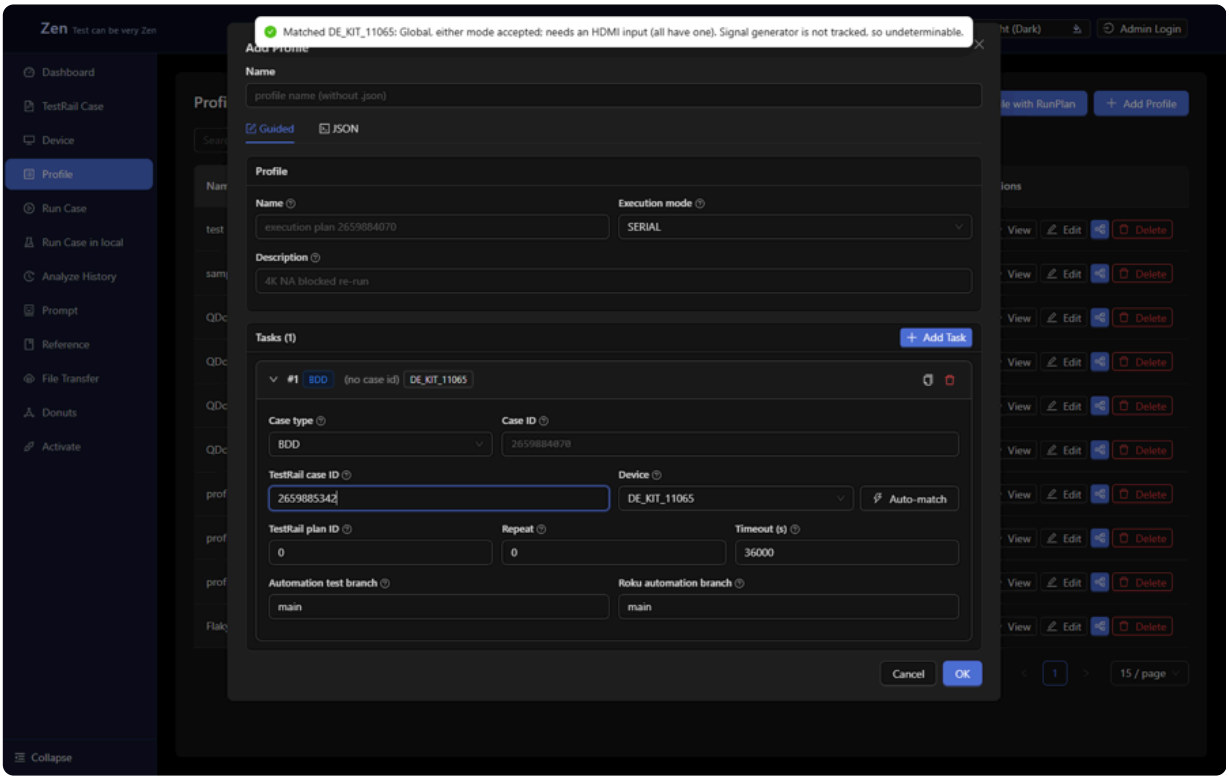
> ⚠ The button stays **disabled until the task has a TestRail case ID** — that id is the only thing Zen can look the execution conditions up by. The *Case ID* field (the automation id) is not used for matching.

3.2.2 Using it

1. Fill in **TestRail case ID** (e.g. `2659885342`). The button becomes clickable.
2. Click **Auto-match**. It shows a spinner while Zen reads the case from TestRail and compares it against **every device** in the Device tab.
3. The **Device** dropdown is filled in with the matched device and a green toast explains *why*.



A successful match looks like this — the device is set to `DE_KIT_11065` and the toast carries the model's reasoning:



What it compares. On the case side: preconditions, test objective, steps, mode/account status (Connected vs Unconnected), region and product area. On the device side: wired-up peripherals (QA Bot, Maxwell, STB, Power, AVR, HDMI 1-3, tuner input) plus live Roku

info (account region, linked/unlinked, connection type, form factor, 4K, supported tuner types).

3.2.3 Good to know

Behaviour	What it means for you
First compatible device wins	Unlike Split, Auto-match does *not* balance load. If several devices qualify you always get the same one, so matching ten tasks can land them all on one device. Spread them yourself, or build the profile first and then use **Split** (§3.3).
It searches every device	The candidate pool is the full Device list, not a selection — including devices that are busy or offline. Check availability before you run.
The JSON tab updates too	Both editor tabs share one JSON document, so the matched device is already in the JSON when you switch over.
Nothing is overwritten on failure	If no device qualifies, the case id is unknown to TestRail, or the AI gateway errors, you get a red toast and the Device field is left exactly as it was.

Message	Cause / fix
Auto-match failed: TestRail case '...' not found	The id does not exist in TestRail. Check it in the TestRail Case tab.
Auto-match failed: No compatible device...	No device meets the case's conditions (often a missing peripheral or the wrong region). The

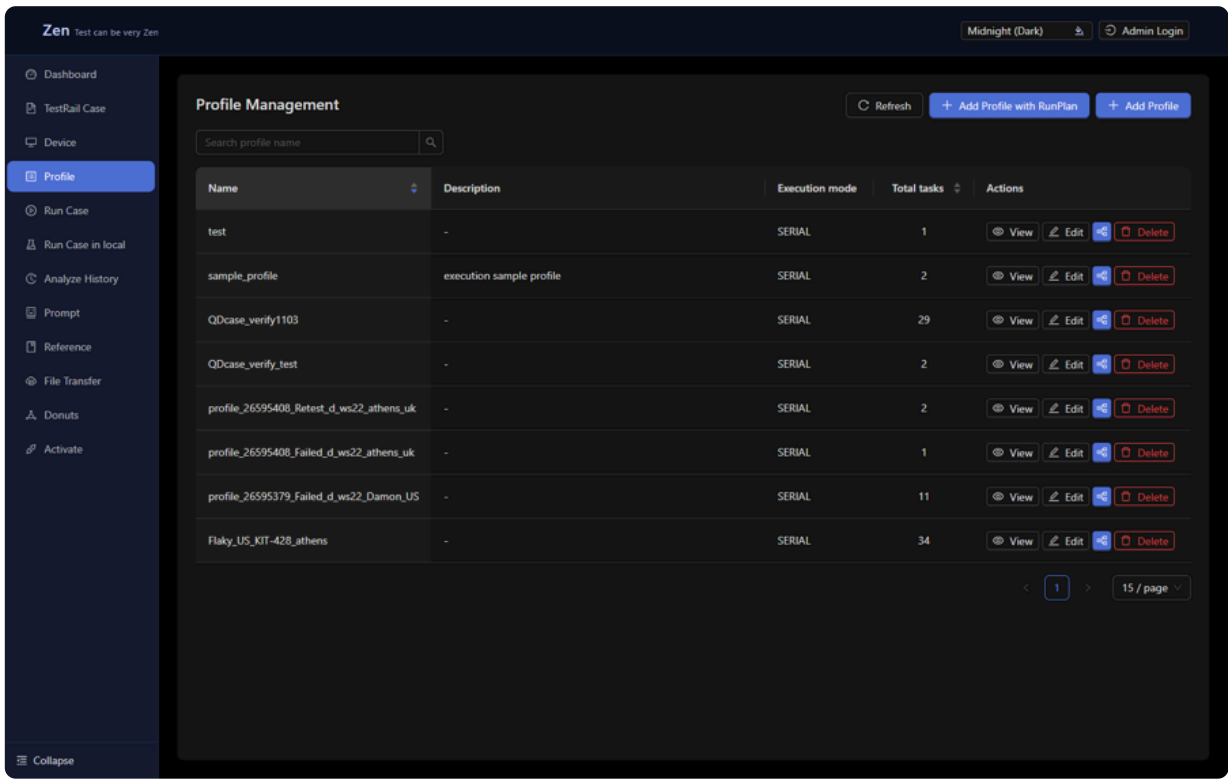
	message carries the model's reason — wire up / pick a device manually.
Auto-match failed: AI analysis failed…	The AI gateway was unreachable. Retry; if it persists check Settings → Agent Settings.

3.3 Split a profile across devices

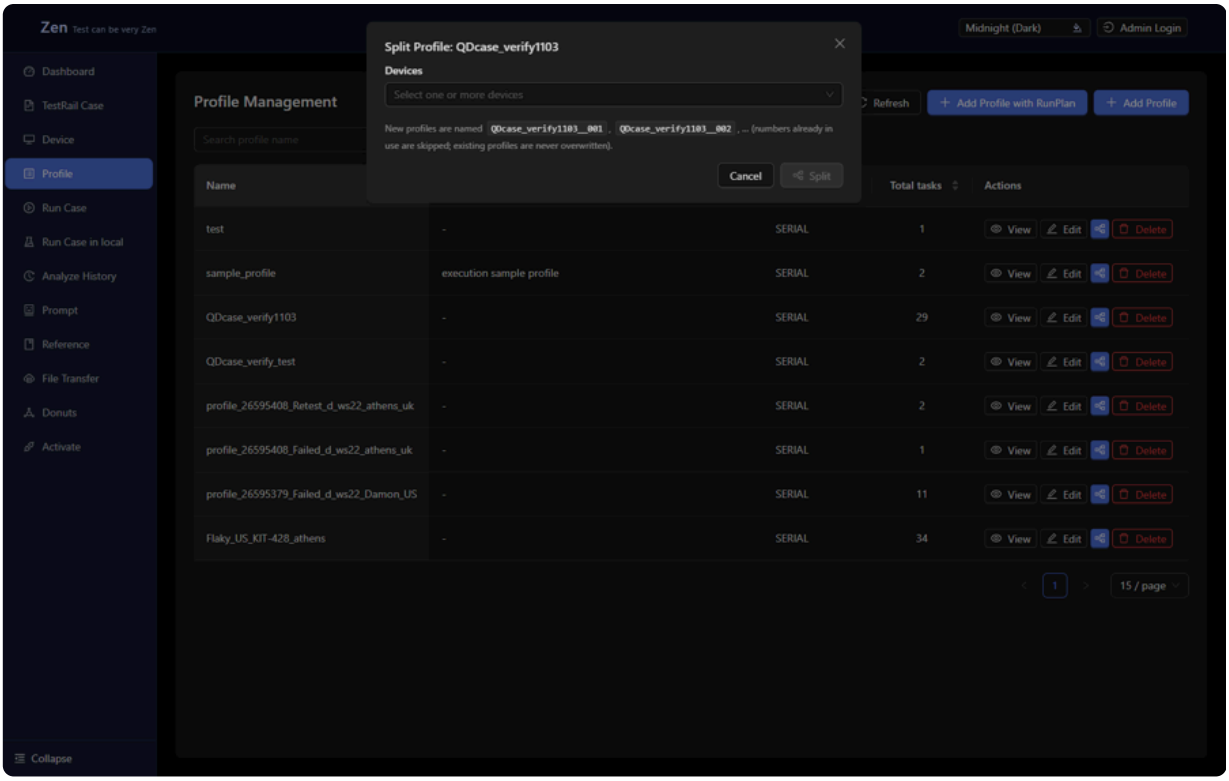
A profile with 30 cases runs for hours on one device. **Split** cuts it into several smaller profiles — one per device — that you can then run at the same time. The cases are not dealt out at random: Zen asks the AI which of the devices you picked can actually *run* each case, and only then balances them.

3.3.1 Open the dialog

On the Profile table, each row has a **Split** action (the ⑂ icon, between *Edit* and *Delete*).

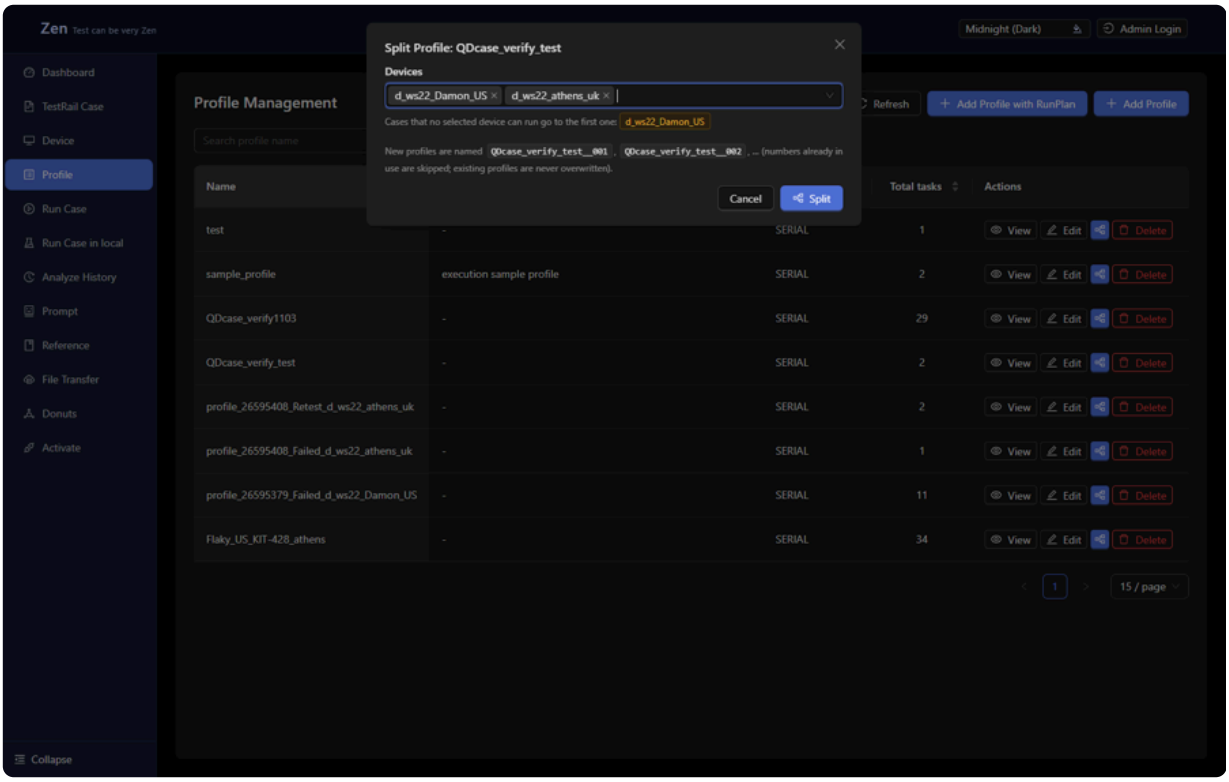


The dialog opens with the target profile in its title and tells you how the new profiles will be named:



3.3.2 Pick the devices

Select **two or more devices**. The list is searchable — type part of a name to filter it.



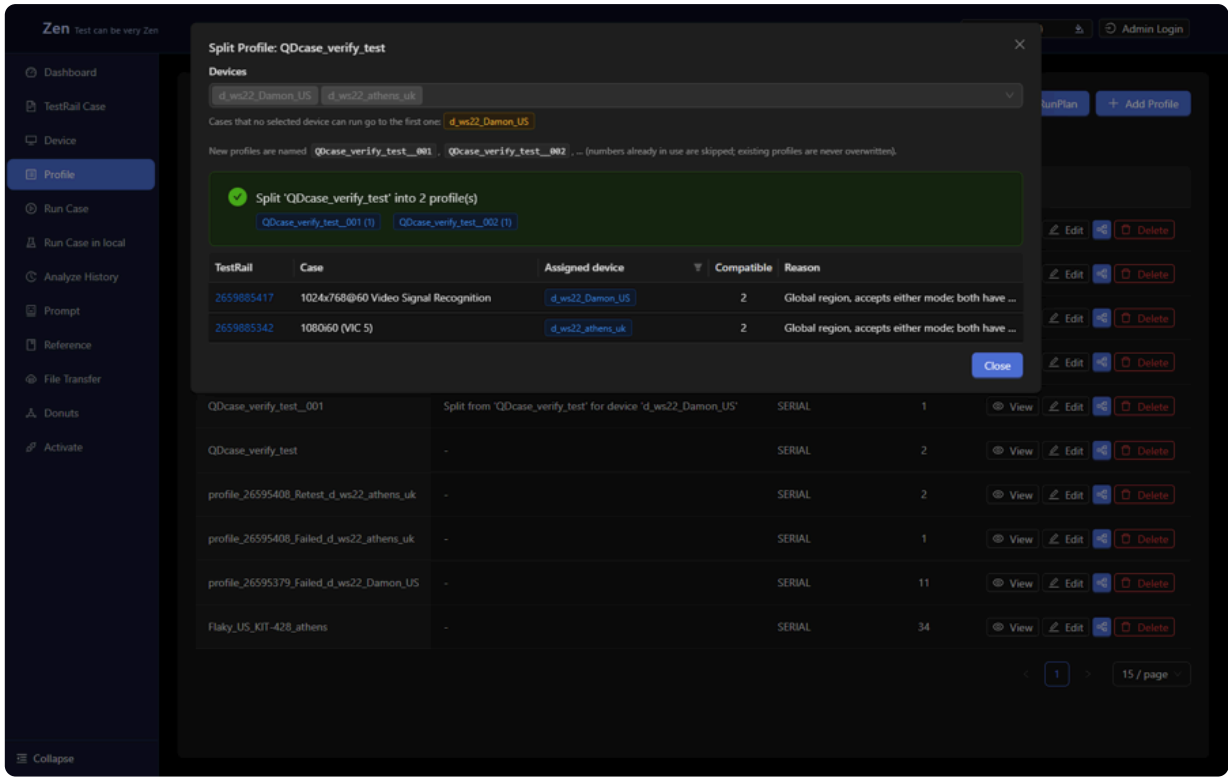
Selection order matters. Any case that *none* of the selected devices can run is placed on the **first device you picked**. The dialog spells this out under the selector ("Cases that no

selected device can run go to the first one: <device>"), so put your most capable / most general device first.

3.3.3 Run the split

Click **Split**. An "Analyzing cases..." banner appears while Zen fetches each case's TestRail conditions and asks the AI to match them. Cases are processed in batches of 10, so a large profile can take a minute or two — leave the dialog open.

3.3.4 Read the result



The green banner lists the profiles that were created with their case counts, and the table explains every single assignment:

Column	Meaning
TestRail	The case id, linked to TestRail.
Case	The case title (hover for the automation id).
Assigned device	Where the case ended up. Filterable by device. Two extra tags may appear: **fallback** (no selected device met the conditions → placed

	on the first device) and **AI failed** (that batch errored → the case was treated as compatible with everything).
Compatible	How many of the selected devices could run it; hover for the names.
Reason	The model's short justification, e.g. *"Global region, accepts either mode; both have HDMI inputs for external generator."*

Behind the dialog the profile list has already refreshed, showing the new `<profile>__001` / `__002` entries with the description *"Split from '<profile>' for device '<device>'"*. Click **Close**, then run them from the Run Case tab.

3.3.5 Good to know

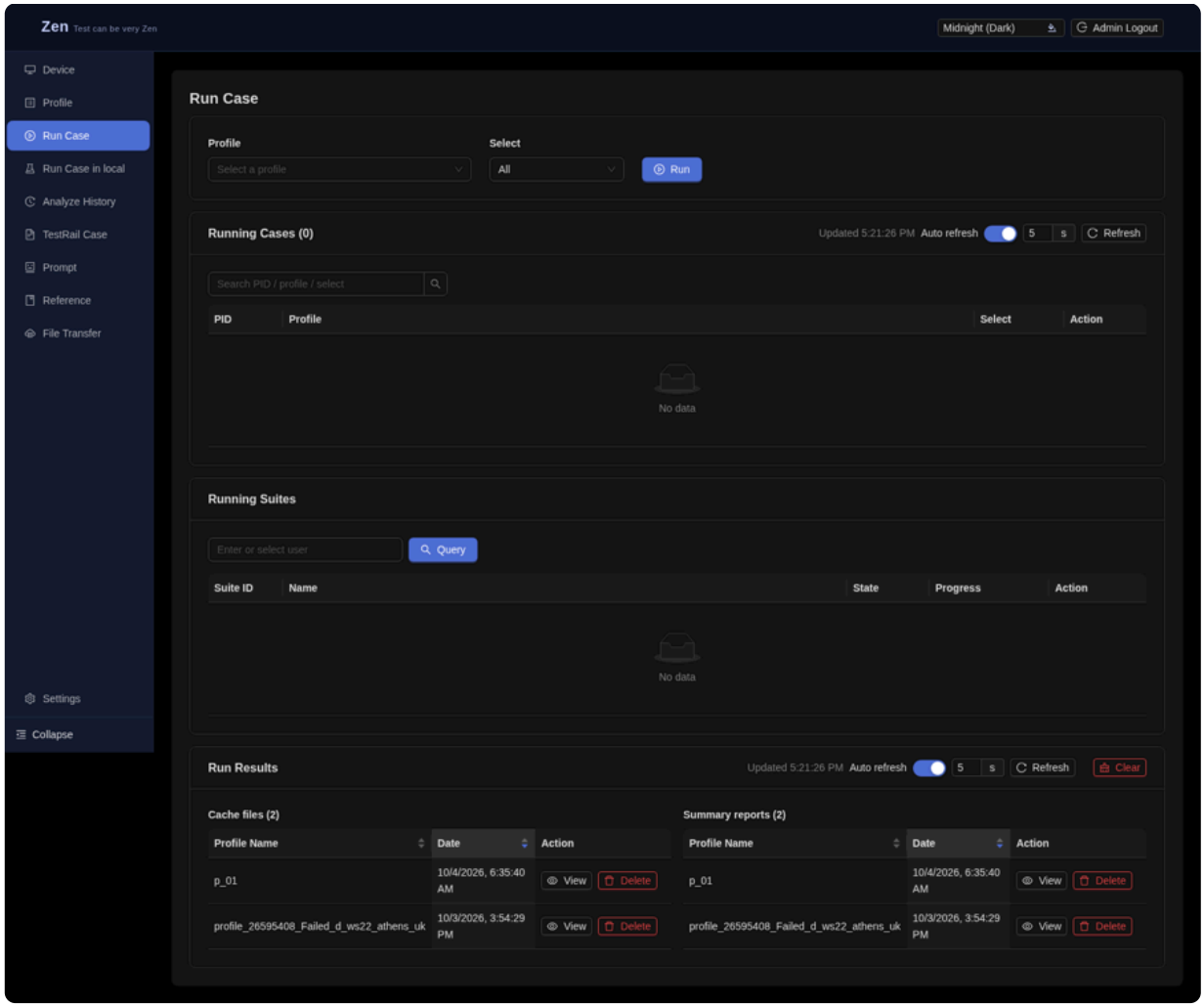
- **The original profile is never touched** — Split only creates new files.
- **Nothing is overwritten.** Numbering starts at `__001` and skips any number already in use, so splitting the same profile twice gives you `__003`, `__004`, … Delete the old ones when you no longer need them.
- **A device with no cases gets no profile** — if you select four devices but only two qualify, only two profiles are written.
- **Load is balanced only among compatible devices.** Zen fills the least-loaded device that can run each case, so counts stay as even as the constraints allow.
- **The matching prompt is configurable** in **Settings → Split Profile Prompt** (§14.2) — the same prompt powers Auto-match.

3.4 Add Profile with RunPlan (generate from TestRail)

1. Click **Add Profile with RunPlan**.
2. **Plan** — only plans with cached TestRail data appear (shown as `id - name`).
3. **Status** — only statuses cached for that plan (e.g. Failed); disabled until a plan is picked.
4. **Device** — the device to run on.
5. **Add** — builds `profile_<plan>_<status>_<device>.json` (one task per case; each case's automation id becomes a BDD/UT config).

If a plan/status is missing, cache it first in the TestRail Case tab (§7.2).

4. Run Case Tab



Launches test profiles and gives three live views: local runner processes, QA Hub suites, and result files.

4.1 Running a profile

Pick a **Profile** and a **Select** subset (All, Fail, Pass, Skipped, Blocked, NA, Unknown, NoTest), optionally tick **Auto analyze**, then **Run**. A banner shows the message and PID.

- **Select** — *All* clears the cache and runs every task; *NoTest* skips cases that already have a cached result; any other value re-runs only the cached tasks with that status (so *Fail* is the usual "re-run the failures" choice).
- **Auto analyze** — when the run finishes, Zen automatically AI-analyzes *every non-passing case* using the configured Analyze prompt and default locale, exactly as if you had clicked Analyze on each row. Results appear in **Analyze History** (§6), where the profile row shows *Analyzing n/m* while the batch is still running.

You do not have to decide up front: the same batch analysis can be started later from the result viewer of a finished run.

4.2 Running Cases (local processes)

Lists running `test_runner.py` processes (PID, Profile, Select, Stop). Toggle **Auto refresh**, set the interval, or **Refresh**.

4.3 Running Suites (QA Hub)

Pick/enter a user, **Query**; the table lists that user's running QA Hub suites: Suite ID, Name (links to the Jenkins build console), State, Progress, Terminate.

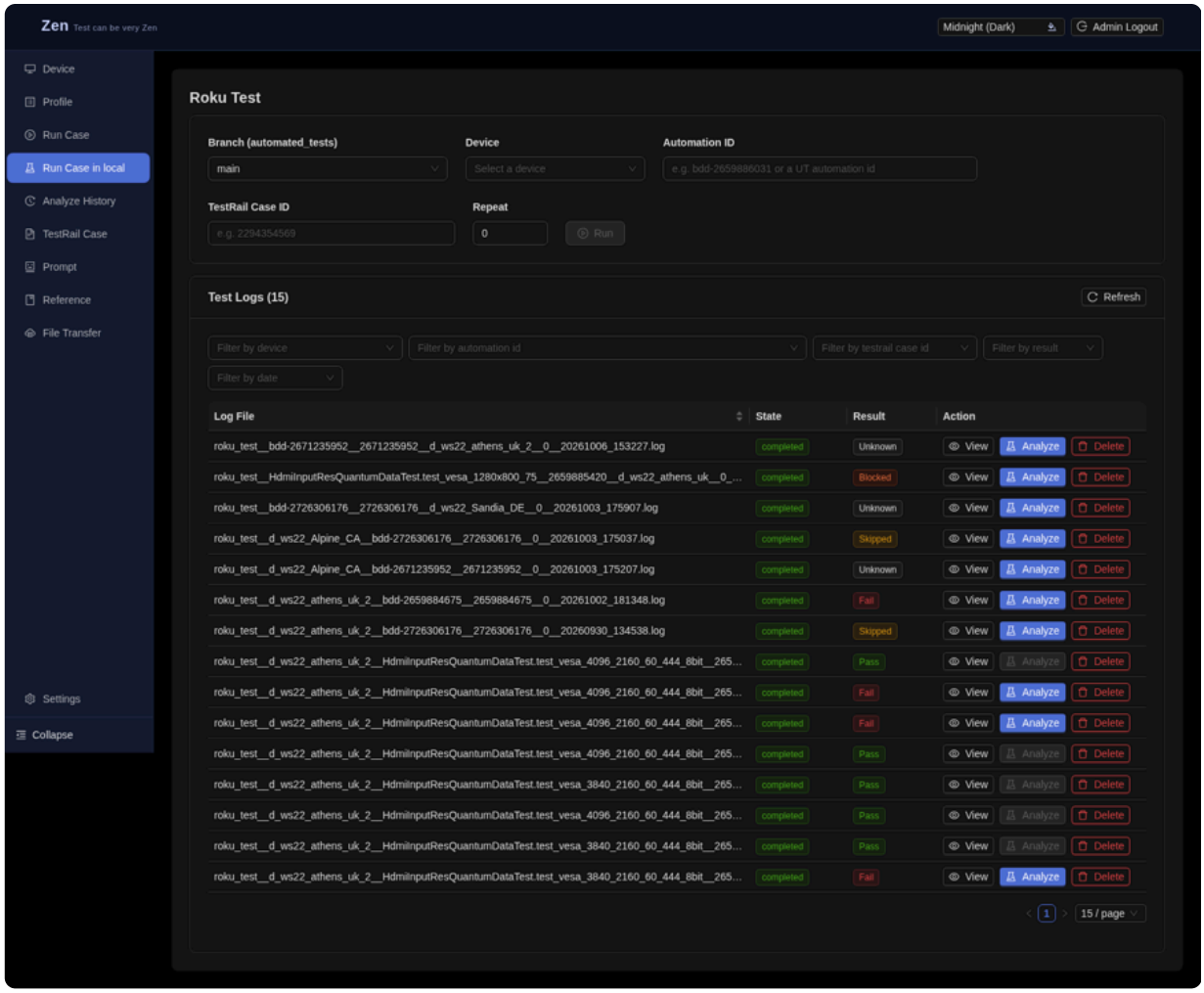
4.4 Run Results (cache & reports)

Two tables (Cache files, Summary reports), each with Profile Name + Date and adjustable refresh. **View** opens a per-run detail table (TestRail Case ID, Automation ID, Status, Build, Log, and an **Analyze** action). **Export Markdown** downloads the table; **Delete** removes a file; **Clear** removes all.

4.5 Analyze a result

Click **Analyze** on a result row to run AI root-cause analysis: Zen fetches that case's Jenkins log server-side and sends it with the configured Analyze prompt template (§14.2). The analysis opens in a Markdown dialog (switch language en/zh to re-run) and is saved to **Analyze History** (§6). Default language and template are set in Settings → Agent Settings.

5. Run Case in local Tab



Runs a single Roku test directly (via the `roku_test` CLI) and tails its log live.

5.1 Running a test

- **Branch (automated_tests)** — pick a git branch; selecting it checks it out on the server.
- **Device**, **Automation ID** (e.g. `bdd-2659886031`), **TestRail Case ID**, **Repeat**.

Run is enabled only when Device, Automation ID and TestRail Case ID are all filled. After starting, a banner shows the PID, live status, and the exact command.

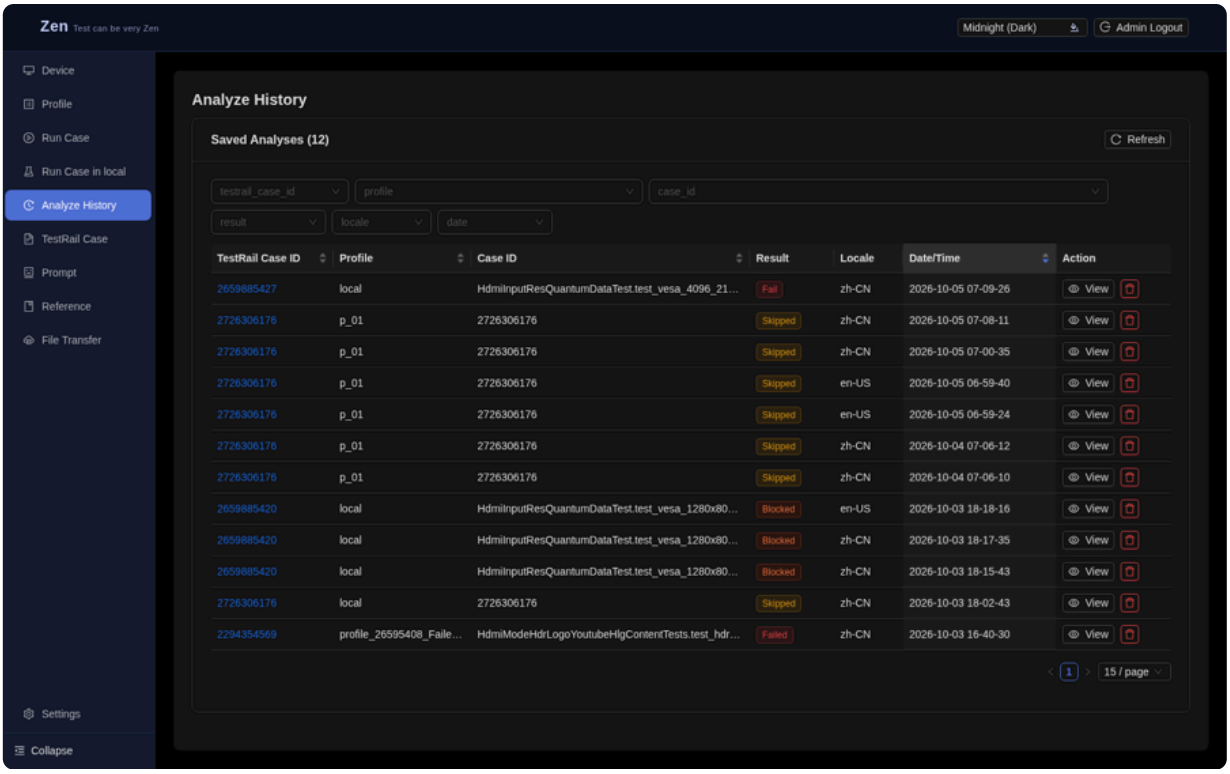
5.2 Live log

Streams the running test output (error lines red, warnings amber), auto-scrolling; survives a page refresh while the test runs.

5.3 Test Logs

Past logs with filters (device, automation id, testrail case id, result, date). Columns: Log File, State, Result (color-coded), Action. **View** opens the log; **Analyze** runs AI analysis (disabled when Pass; tagged with profile `local` in Analyze History); **Delete** removes it.

6. Analyze History Tab



This tab has two tables: the per-profile roll-up reports produced by **Auto analyze** (§4.1), and every individual AI analysis saved from Run Case (§4.5) or Run Case in local (§5.3).

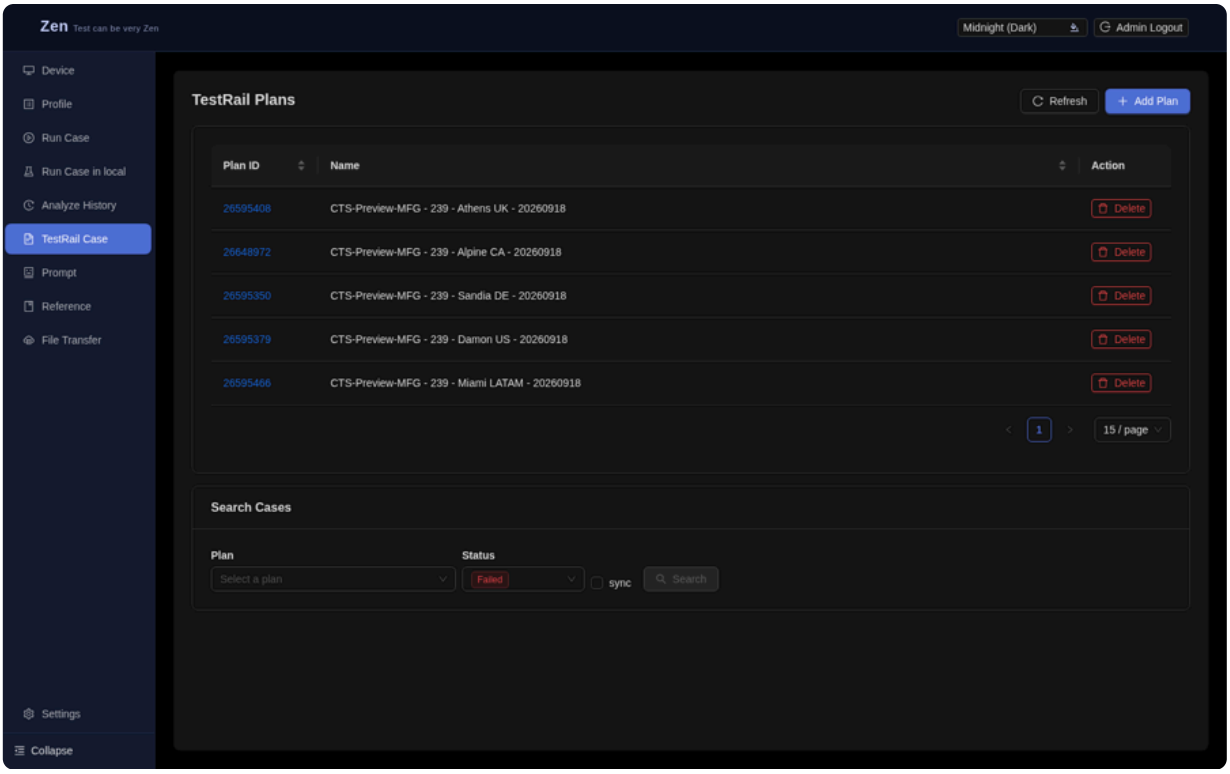
Profile Analyze Reports — one row per analyzed run:

- **Filters**: Profile, Locale, Date.
- **Columns**: Profile Name, **NoPass Cases** (red when > 0), Total Cases, Locale, Date/Time, **Case Report**, Action.
- **Case Report** shows a spinner and *Analyzing n/m* while the batch is still running; when it completes, each non-passing case becomes a link that opens its analysis in the Markdown viewer.
- **Delete** removes the roll-up report only; the individual case analyses stay in the table below.

Saved Analyses — every individual analysis:

- **Filters**: TestRail Case ID, Profile, Case ID, Result, Locale, Date.
- **Columns**: TestRail Case ID (links to TestRail when numeric), Profile, Case ID, Result, Locale, Date/Time.
- **Actions**: **View** (renders Markdown), **Delete**.

7. TestRail Case Tab



Manage TestRail plans and search their cases by status; results are cached to feed the profile generator.

7.1 Plans

The Plans table lists saved plans (Plan ID links to TestRail, Name, Delete). **Add Plan** with the Plan ID (e.g. `26595408`) and a Name.

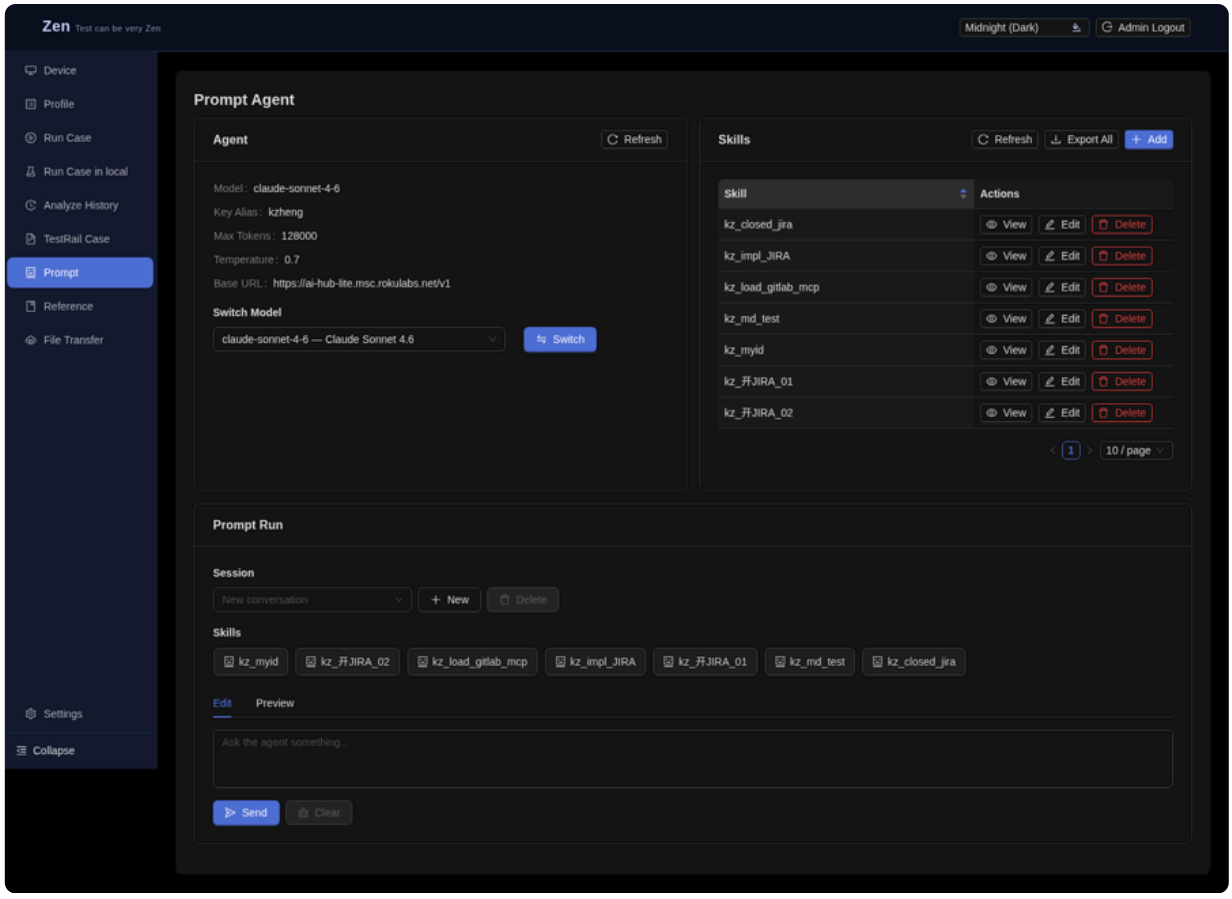
7.2 Searching cases

In **Search Cases**, pick a **Plan** and **Status** (colored tags), optionally tick **sync** to bypass the cache, then **Search**. Columns: Case ID (links to TestRail), Title, Automation ID, Status, Run ID.

7.3 Feeding the profile generator

A search caches results as `<plan>_<status>.json`; that plan+status then becomes selectable in **Profile → Add Profile with RunPlan** (§3.2).

8. Prompt Tab (AI Assistant)



An AI assistant that answers using Roku internal data (JIRA, Confluence, TestRail, GitLab) via retrieval-augmented generation (RAG), with multi-turn sessions and reusable Skills.

8.1 Agent card — model info & switching

Shows the active Model, Key Alias, Max Tokens, Temperature, Base URL. Pick a model from **Switch Model** and **Switch**. The model list is managed in Settings (§14.2).

8.2 Sessions (multi-turn chat)

The **Session** bar: **Select** a saved session (loads its history), **New**, or **Delete**. Each session keeps its messages so the assistant has context; the title comes from the first prompt.

8.3 Skills

A **Skill** is a saved Markdown fragment. The Skills card lists them with **View**, **Edit**, **Delete**, **Add**, and **Export All** (downloads the whole skill library as a zip). Each skill also appears as a clickable **chip** in Prompt Run to append its content.

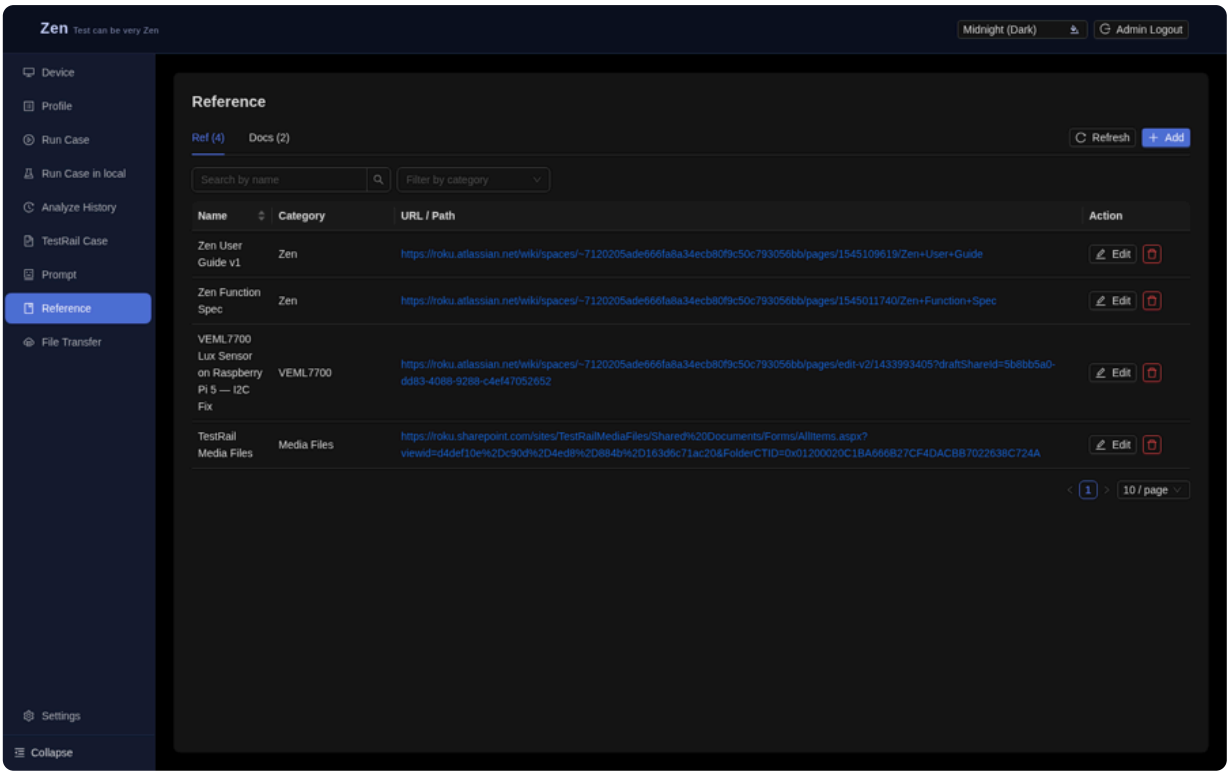
8.4 Prompt Run

Type your question (Edit/Preview Markdown tabs; click skill chips to prepend fragments) and **Send**. Zen runs with internal-data access (JQL path then semantic fallback). **Clear** resets; the answer renders as Markdown with **Save** (.md) and **Copy**.

8.5 Search card

Direct semantic search over indexed internal data: enter a query, pick **Sources** (JIRA, Confluence, TestRail, GitLab, File, S3, Web), **Search**; results list the Document (linked) and a preview.

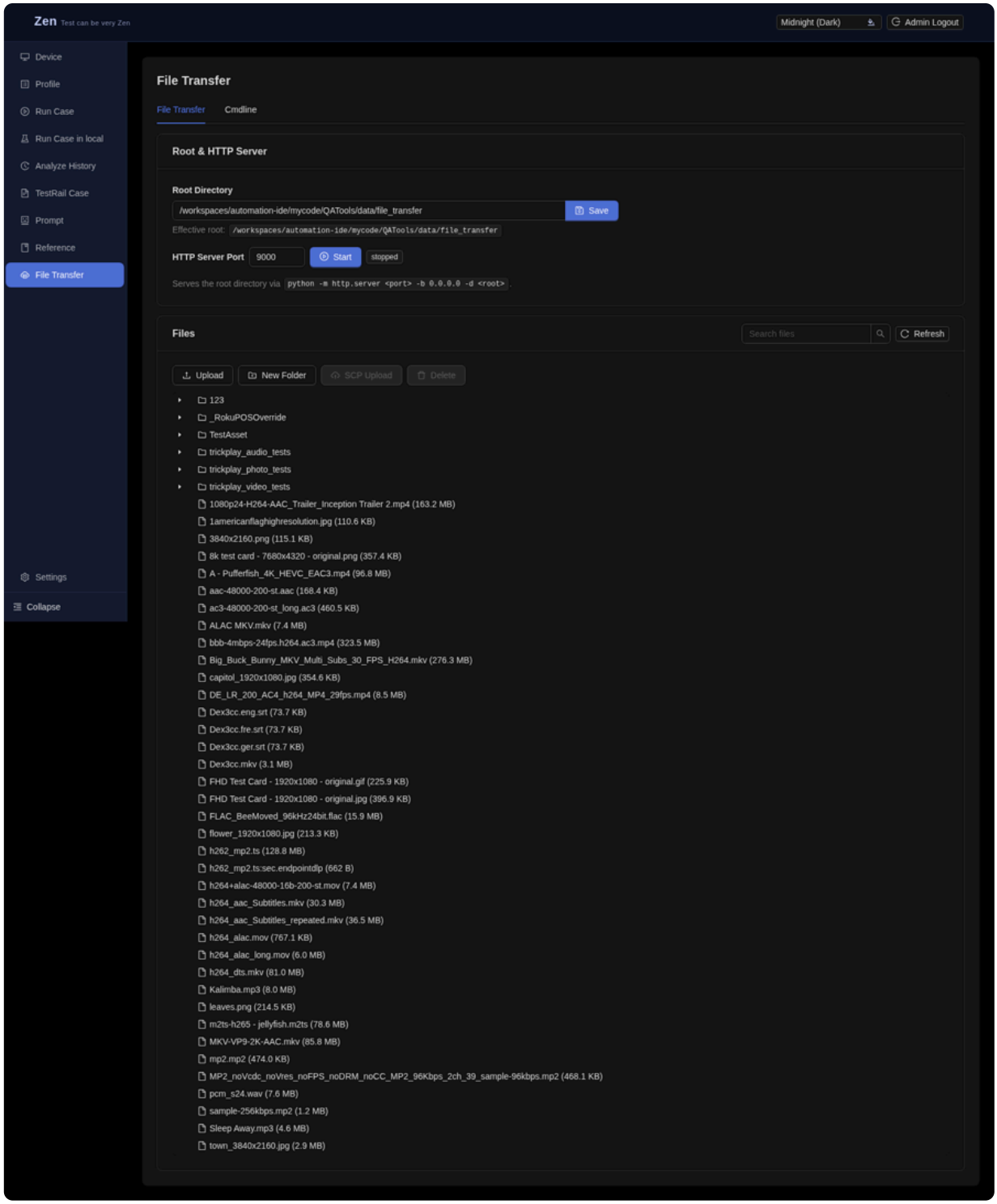
9. Reference Tab



A small team library with two sub-tabs:

- **Ref** — reference links; each entry has Name, Category, URL (clickable). Add/Edit/Delete; filter by name or category.
- **Docs** — uploaded Markdown docs; **Add** accepts a `.md` file (same name overwrites), **View** renders it. Deleting a doc record keeps the underlying file.

10. File Transfer Tab



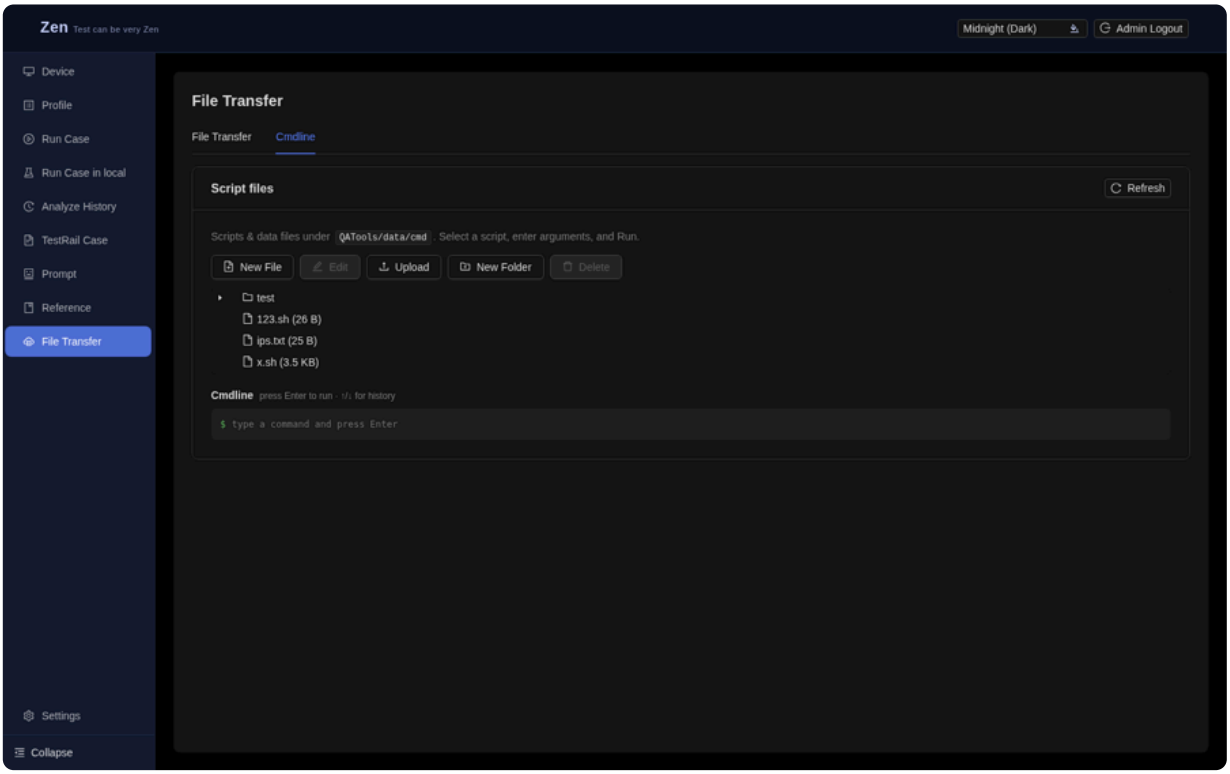
Two sub-tabs: **File Transfer** and **Cmdline**.

10.1 File Transfer sub-tab

- **Root & HTTP Server** — edit the managed **Root Directory** (default `QATools/data/file_transfer`) and **Save**; set the **HTTP Server Port** (default 9000) and **Start** / **Stop** a `python -m http.server` that serves the root; when running, a live clickable URL is shown.

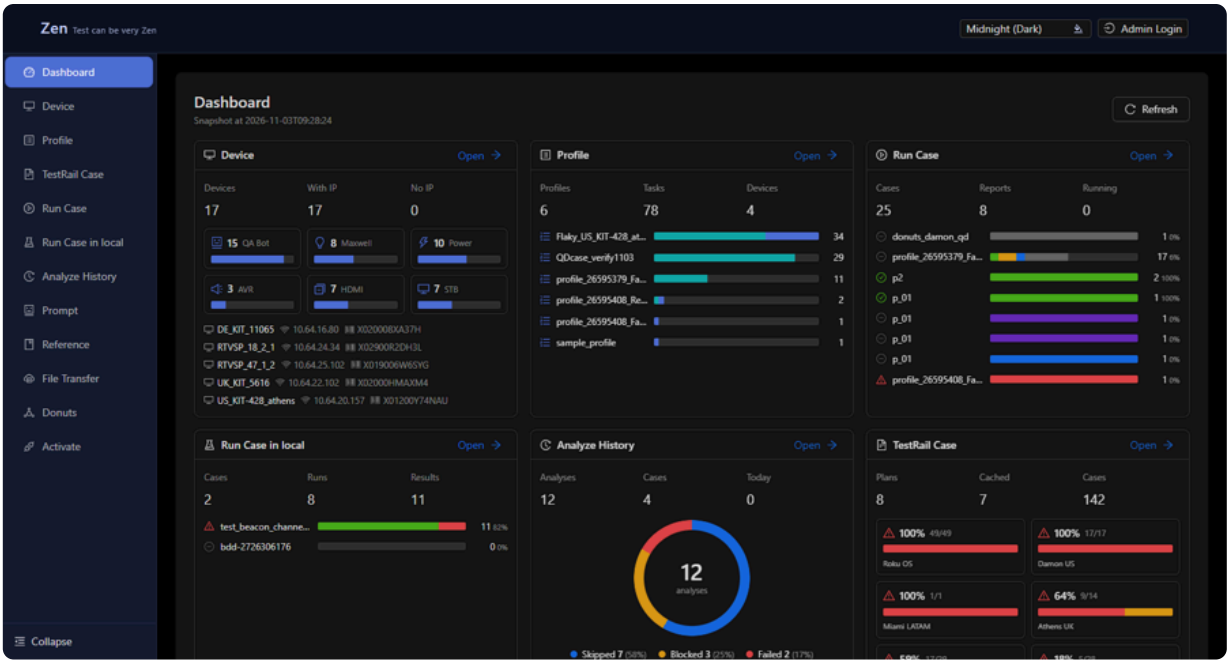
- **Files** — a file tree with **Search files**, **Upload** (with a real progress bar), **New Folder**, **Delete**, and **SCP Upload**.
- **SCP Upload** — a modal where **Host** is an autocomplete of previously-used servers (picking one autofills Remote Directory + User; each remembered host can be deleted). Enter Password; a live progress bar tracks the transfer. Successful targets are remembered.

10.2 Cmdline sub-tab



- **Script files** — a tree of scripts/data under `QATools/data/cmd`: **New File** (opens a shell-highlighted editor), **Edit**, **Upload**, **New Folder**, **Delete**. Clicking a file inserts its path into the command line.
- **Terminal console** — type a command and press **Enter** to run; output appears inline (stderr in red, with an `[exit N]` footer). Empty Enter adds a blank prompt line; **↑/↓** recall history; the working directory persists across commands (so `cd ..` works). `ssh` / `scp` with a remote command prompt for a password; a bare `ssh user@host` opens a **persistent interactive SSH session** (the prompt turns amber with `user@host $`; type `exit` to disconnect).

11. Dashboard Tab

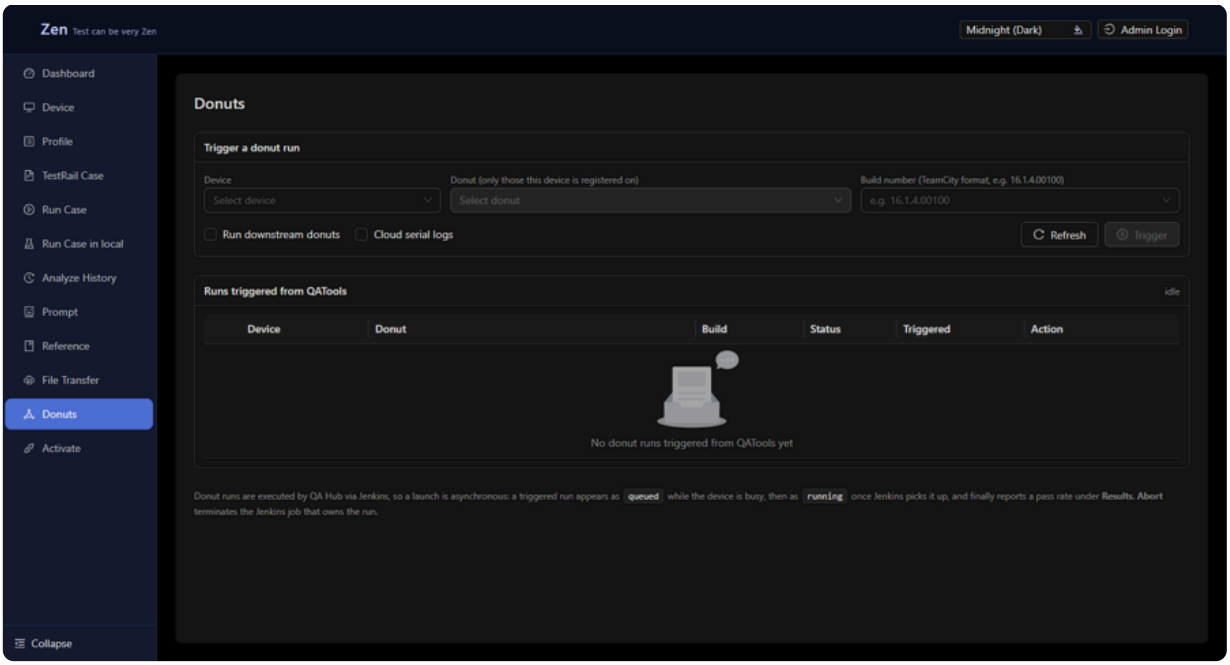


The Dashboard is the landing page. It gives a read-only summary of the workspace with one card per tab. The page does not change anything; cards only navigate.

11.1 Controls

- **Refresh** (top right): reloads all cards. Use it after you change something in another tab.
- **Snapshot time**: shown next to the title; it is when the data was read.
- **Click a card**: switches to that tab.
- Only tabs that are visible in Settings → Tab Settings (§14.1) have a card.
- If one card cannot load, it shows its own error and the other cards still show data.

12. Donuts Tab



Start a QA Hub donut run on a device and follow it from here. The page has two parts: the **Trigger a donut run** card and the **Runs triggered from QATools** table.

12.1 Trigger a donut run

1. **Device**: choose a device from the list (type to search). Its serial number is shown under the box.
2. **Donut (only those this device is registered on)**: the list is loaded from QA Hub for that device, shown as *donut · product · branch*. It stays disabled until a device is chosen. If it is empty, the device is not registered on any donut.
3. After you pick a donut, a blue line reads *Will run <donut> on serial <serial> (<platform>)* with the product and branch. Check it before you continue.
4. **Build number**: type a TeamCity-format build number such as `16.1.4.00100` and press Enter. Typing a new one replaces the old one.
5. **Run downstream donuts** and **Cloud serial logs**: optional checkboxes.
6. **Trigger**: enabled only when a device, a donut and a build number are all set.

If the device is busy, QA Hub queues the run. It then appears in the table with status *queued*.

12.2 Runs table

Lists the runs started from QATools, newest first, 10 per page. The card header says *auto-refreshing every 10s* while runs are active, or *idle* otherwise. Click **Refresh** to reload now.

- **Device** and **Donut** (product / branch shown under the name).
- **Build**: the build number you entered.
- **Status**: *running* (blue), *queued* (orange), *finished* (green).
- **Triggered**: when the request was sent.
- **Action**: **Results** and a trash icon.

Each row expands to show what QA Hub is doing with it:

- **Running Jenkins job**: job name, build number and a *running* tag, with a **Terminate** button. Terminate asks *Terminate this donut job?* before it stops the Jenkins build.
- **Queued request**: donut name and *queued id*. Hover over the short detail text to read the full message. A red tag means QA Hub needs attention. Click **Cancel** and confirm *Cancel this queued run?*.
- If neither applies, the row says it has finished (use Results) or has not reached Jenkins yet.

12.3 Results

Click **Results** to open *Results — <donut>*. It shows the device, serial, donut and build, then one line per result:

- **Pass %**: green at 95% or more, orange at 80% or more, red below 80%.
- **Stage**: green when *complete*, blue otherwise.
- **Re-ran failures**: yes or no.
- **Logs**: opens the QA Hub log page in a new tab.

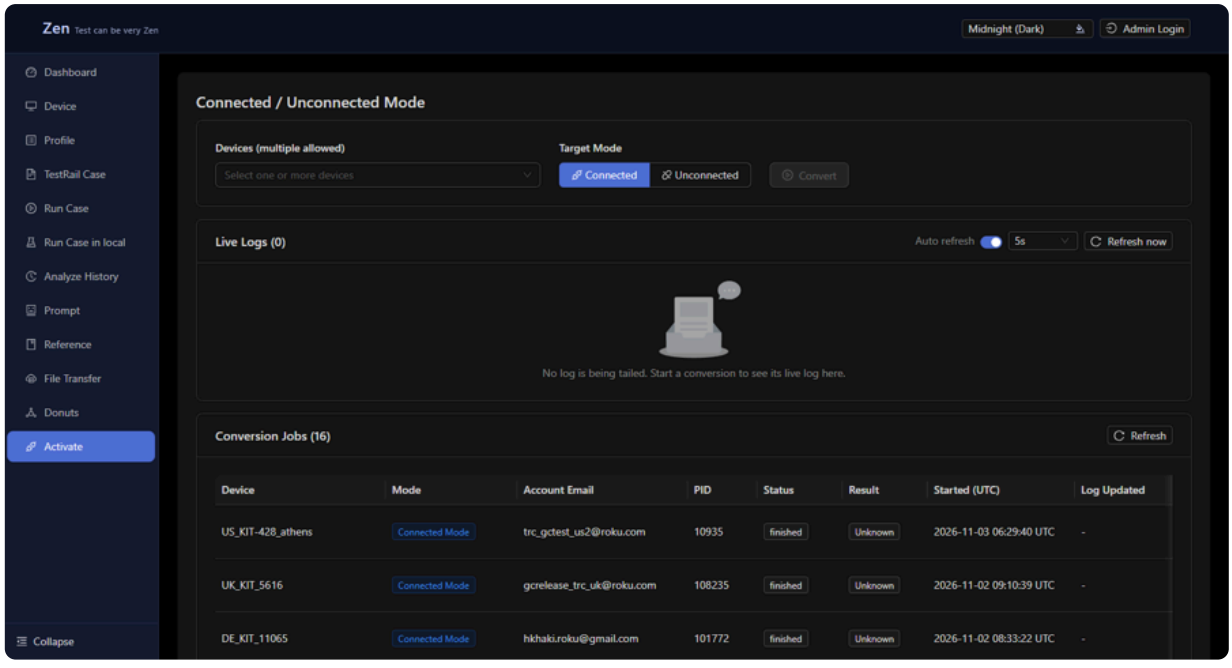
Close the window with the X in its corner.

12.4 Remove a run

Click the trash icon and confirm *Remove from history?*. This deletes only the local record in QATools; QA Hub is not changed.

An empty table shows *No donut runs triggered from QATools yet*. Runs started directly in QA Hub are not listed.

13. Activate Tab



Convert one or more devices to **Connected Mode** (activated and linked to an account) or **Unconnected Mode** (setup finished with “connect to network later”). The page has the conversion controls, the precheck result, **Live Logs** and **Conversion Jobs**.

13.1 Start a conversion

1. **Devices (multiple allowed)**: choose one or more devices. Devices convert in parallel.
2. **Target Mode**: **Connected** or **Unconnected**.

3. In Connected mode, an **Activation Account Email** table appears with one row per device. The email is pre-filled from Settings → Activate Account (§14.4) and can be edited for this run. A field turns red if the address is not valid, and Convert stays disabled until all addresses are valid.
4. **Convert**: a confirmation asks *Convert <n> device(s) to <mode>?* and notes that a precheck runs first and each passing device will be factory reset. The button shows *Prechecking…* while it runs.

13.2 Precheck result

A table shows each device with its **IP**, **Connection** (Wi-Fi/wireless in orange, wired in blue), **Account Linked** (True in green, False in grey), **Allowed** (Yes in green, No in red) and **Reason**.

A wireless device **cannot** be converted to Unconnected Mode, because the factory reset would drop its only connection. Switch it to a wired connection first, then run Convert again.

Only allowed devices start. If any device is refused, a warning reads *N device(s) were not started* and lists each reason. A device that already has a running conversion is refused too.

13.3 Live Logs

Each started conversion opens its own tab. The card title shows how many logs are open, for example *Live Logs (2)*.

- **Tabs**: click a device tab to switch logs. A tab shows *running* while its conversion is active.
- **Colours**: lines containing ERROR, FATAL, Traceback, Exception or FAIL are red; lines containing WARNING are orange.
- **Auto refresh**: the switch turns refreshing on or off. Choose the interval from 1s, 2s, 3s, 5s (default), 10s or 30s. The setting is remembered in this browser.
- **Refresh now**: loads the newest lines immediately.
- **Scrolling**: the view follows new lines while you are at the bottom; if you scroll up, it stays where you are.

13.4 Conversion Jobs

Lists every conversion started from QATools, including finished ones. The card title shows the total and the number running, for example *Conversion Jobs (5, 2 running)*. Click **Refresh** to reload.

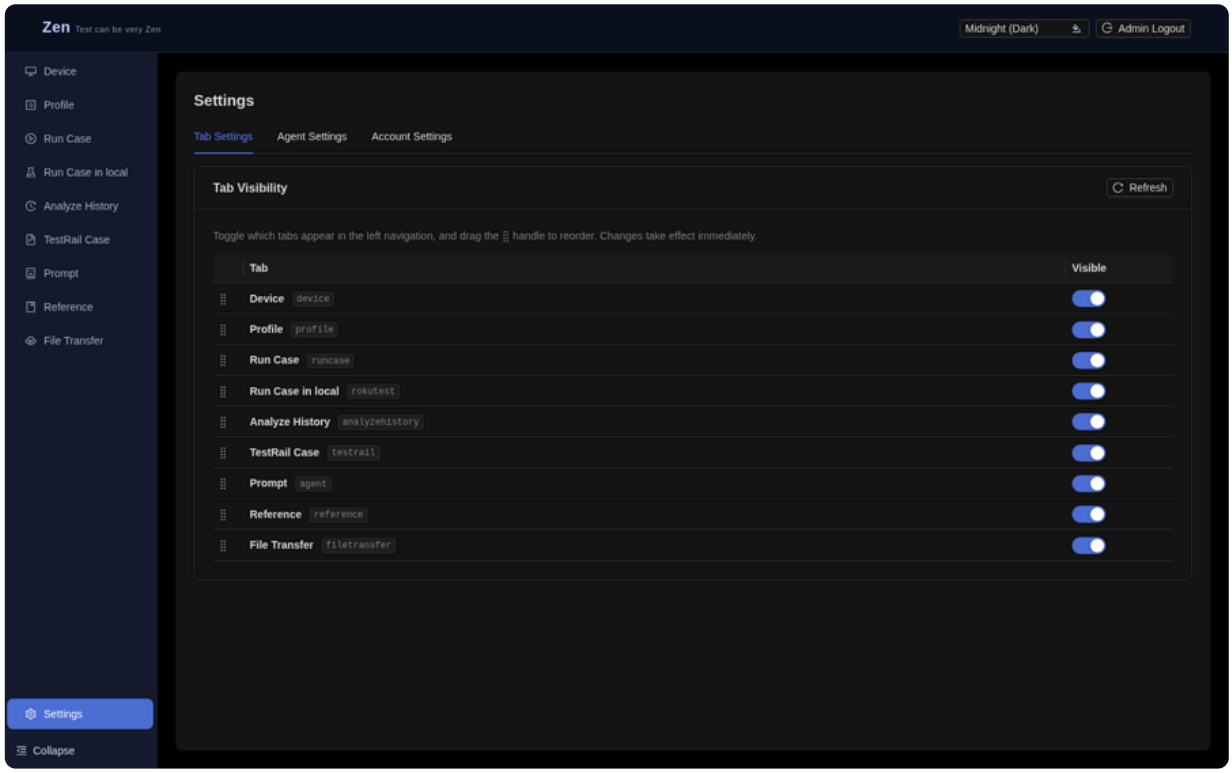
- **Device**, **Mode** (Connected in blue, Unconnected in purple), **Account Email** (Connected only).
- **PID**, **Status** (*running* while the process is alive).
- **Result**: *Success* (green) or *Failed* (red), as reported by the job; blank while it runs.
- **Started (UTC)** and **Log Updated**.

Actions:

- **View log** (eye icon): opens the log in a dark window. Close it with the X.
- **Stop** (stop icon): enabled only while the job runs. Confirm with *Stop PID <n>?* and the **Stop** button.
- **Delete** (trash icon): removes a finished job from the list. Disabled while the job runs. Confirm with **Remove**.

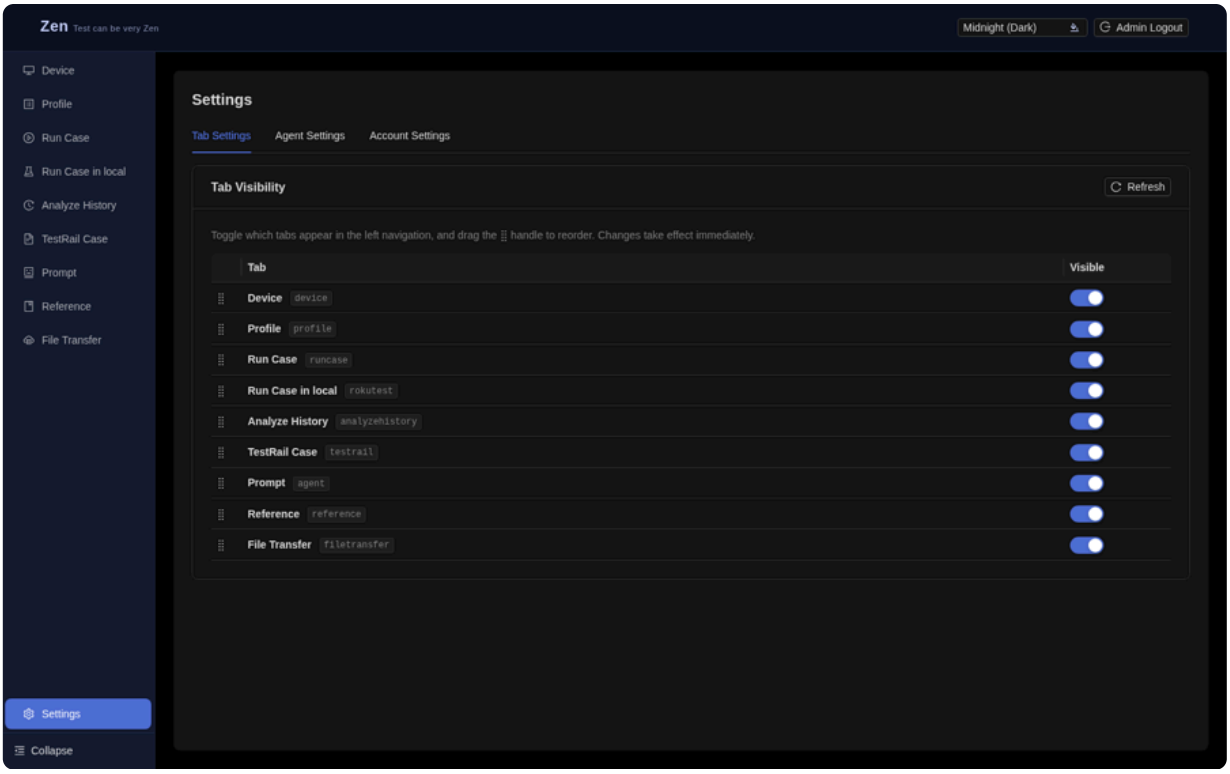
A full conversion takes several minutes. Activation waits up to 10 minutes for the account link.

14. Settings Tab (admin only)



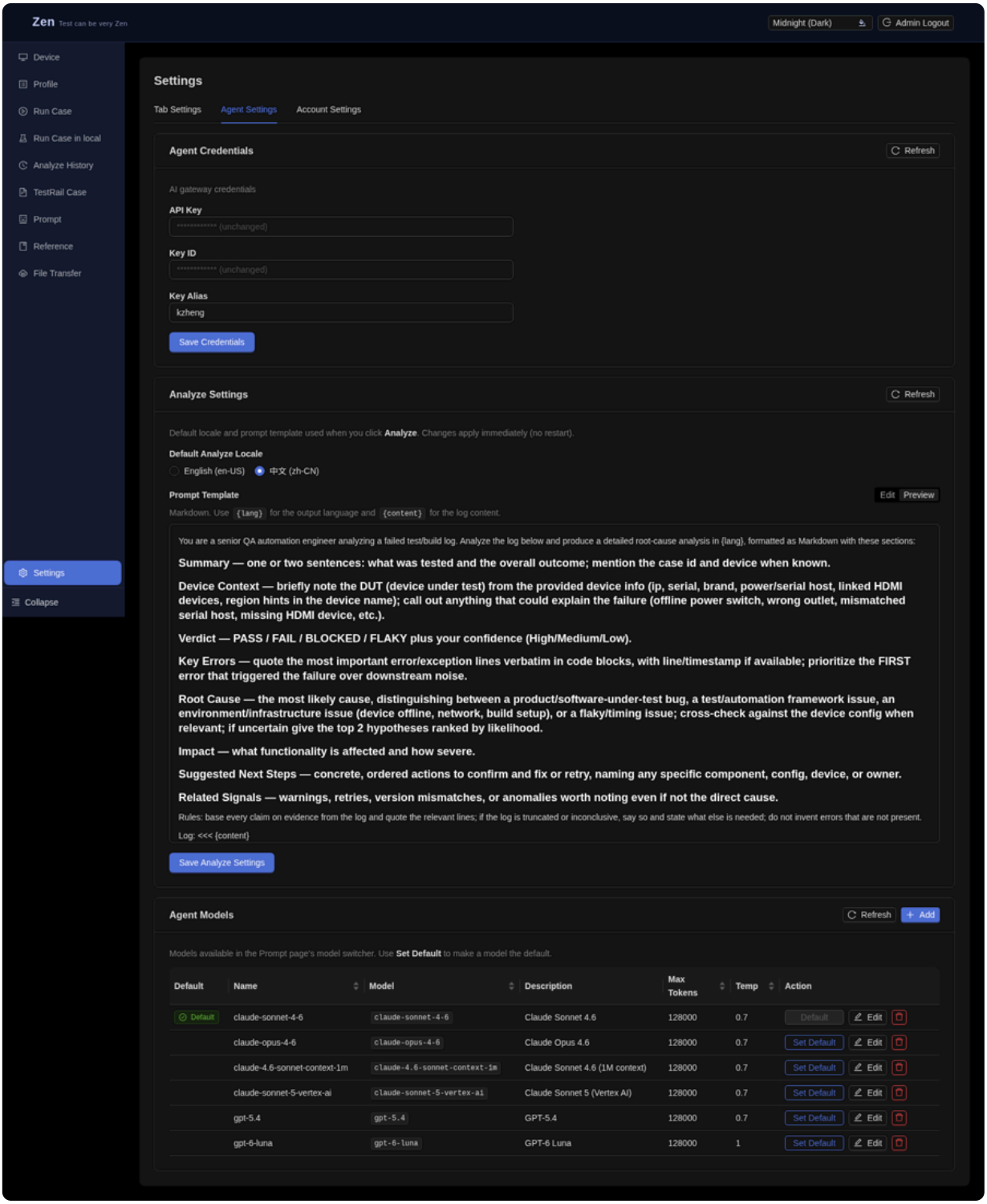
The **Settings** tab is only visible after **Admin Login** (§1.4). It is where the whole system is configured, across its sub-tabs (Tab Settings, Agent Settings, Environment Settings, Activate Account). Some changes apply live; credential/account changes require a backend restart (Zen offers to restart for you).

14.1 Tab Settings



Tab Visibility — a table of all tabs with a drag handle (⁝), the tab label + key, and a **Visible** switch. Toggle to show/hide a tab, or drag rows to reorder them; both apply to the left navigation immediately. (Settings itself is always admin-gated and not in this list.)

14.2 Agent Settings



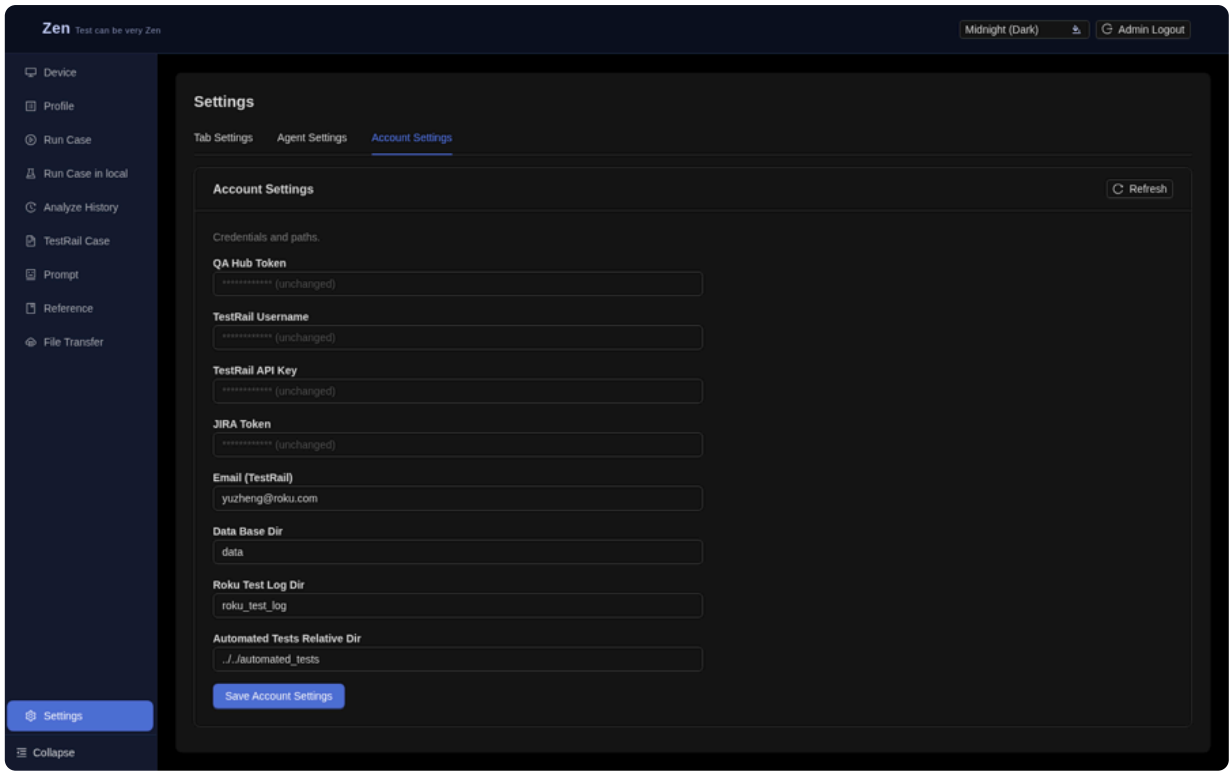
Controls the AI used by the Prompt tab and all Analyze actions:

- **Agent Credentials** — the AI gateway auth: **API Key** and **Key ID** (write-only/masked — leave blank to keep the stored value) and **Key Alias** (identifies the caller). Saving offers a backend restart to apply.
- **Analyze Settings** — the **Default Analyze Locale** (English `en-US`, the shipped default / 中文 `zh-CN`) used when you click Analyze or enable Auto analyze, and the **Prompt Template** sent

to the AI (Edit / Preview Markdown toggle). The template supports `{lang}` (output language) and `{content}` (the log). Applies immediately — no restart.

- **Split Profile Prompt** — the prompt used by Profile → **Split** and **Auto-match** (§3.2, §3.3). Same Edit / Preview editor and **Reset to Default** button as the Analyze template. It must keep the `{devices}` and `{cases}` placeholders — Save is blocked while either is missing, and a saved template that later loses one is ignored in favour of the built-in default. Applies to the next split; no restart.
- **Agent Models** — the models offered in the Prompt tab's Switch Model dropdown. Add / Edit / Delete a model (Name, Model identifier, Description, Max Tokens, Temperature) and mark one as the **Default** (applied live). The default is the model Zen starts with and uses for Analyze.

14.3 Environment Settings

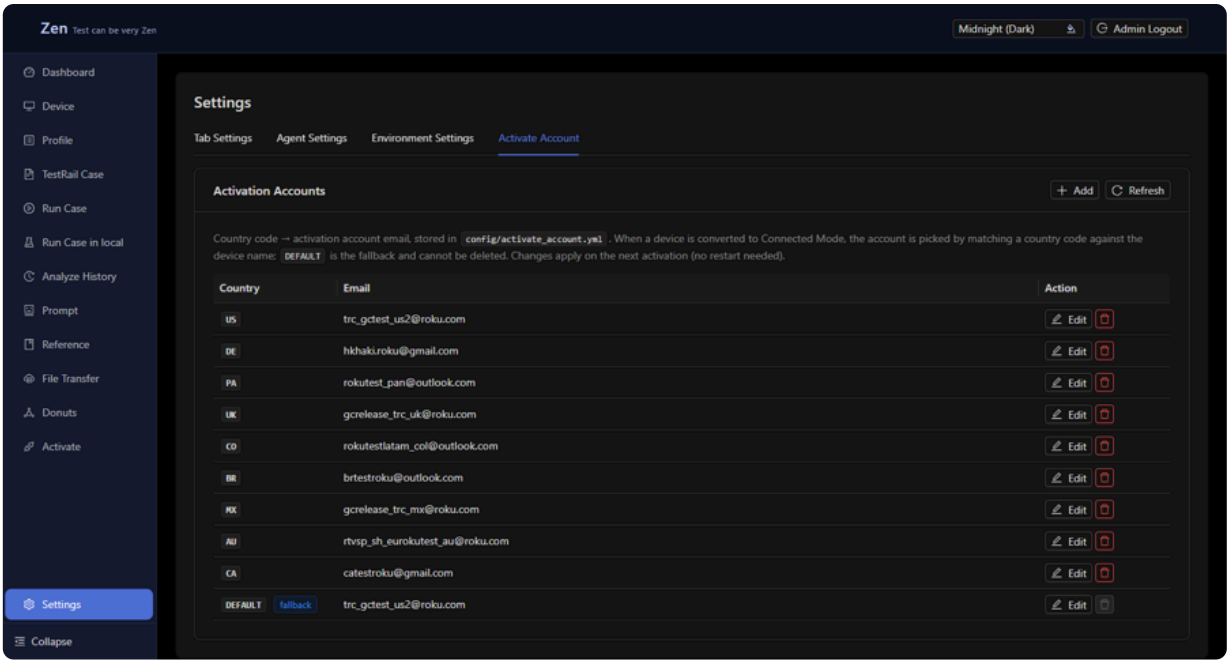


Edits the credentials and paths that live in `config/env.py` — that file *is* the source of truth and its assignments are rewritten in place. **Secret fields are masked**; leave a secret blank to keep the stored value. **Changes take effect after a backend restart** (Zen offers to restart on save).

Field	Secret	What it controls
QA Hub Token	Yes	Auth for all QA Hub calls (device sync, suite list/terminate).

TestRail Username	Yes	TestRail API username (with the API key) for plan/case fetch.
TestRail API Key	Yes	TestRail API key (Basic auth with Email).
JIRA Token	Yes	JIRA access used by the assistant's internal search.
Email (TestRail)	No	TestRail login email (Basic-auth username).
Automated Tests Relative Dir	No	Location of the `automated_tests` git repo (branch list/checkout, roku_test target).
Roku Automation Relative Dir	No	Location of the `roku_automation` repo used by the `roku_automation_branch` field of a profile config.

14.4 Activate Account



Maps a **country code** to the activation email used by Connected-mode conversions. The data is stored in `config/activate_account.yml`.

1. Open **Settings** (admin login required, §1.4) and choose the **Activate Account** sub-tab.
2. The table shows **Country**, **Email** and **Action** for each entry.
3. **Add**: opens *Add Activation Account*. Enter a **Country Code** (for example `US`, `UK`, `DE`; saved in upper case) and the email, then click **Save**.
4. **Edit** (pencil icon): opens *Edit Activation Account: <country>*. The country code cannot be changed; edit the email and click **Save**.
5. **Delete** (trash icon): removes the entry after confirmation. The **DEFAULT** entry cannot be deleted.

When a device is converted, Zen looks for the first country code that appears in its name (for example `DE_KIT_11065` matches `DE`). If none matches, the DEFAULT email is used. Changes apply to the next conversion; no restart is needed.

15. Tips & Troubleshooting

- **Settings tab missing** — you are not logged in as admin. Click **Admin Login** (default `admin` / `Zaq1@wsx`). A page reload logs you out.
- **Save / Copy** work best in Chrome/Edge (native file picker + clipboard); other browsers fall back to a direct download.
- **"Add Profile with RunPlan" shows no data** — cache that plan+status first in the TestRail Case tab (§7.2).

- **Analyze says the prompt template is not configured** — set it in Settings → Agent Settings → Analyze Settings (§14.2).
- **A tab is missing** — it was hidden in Settings → Tab Settings; re-enable it there.
- **Account/credential change didn't take effect** — those apply after a backend restart; accept the restart prompt on save.
- **Donut list is empty for a device** — the device is not registered on any donut in QA Hub, or its configuration has no serial number.
- **Activate refuses a device** — read the Reason column. Usual causes: a wireless device with Unconnected Mode selected, or a conversion already running on it.